SPPU TE BE COMP Click to join telegram

---

1. Define linked list and types.

Linked List

Definition

A **linked list** is a **dynamic data structure** in which elements (called **nodes**) are stored in **non-contiguous memory locations**. Each node contains:

- **Data part** – stores the value
- **Link part** – stores the address of the next (and/or previous) node

Nodes are connected using **pointers**, forming a sequence.

Basic Structure of a Node

```
[ Data | Link ]
```

Why Linked List?

- Dynamic size (can grow or shrink at runtime)
- Efficient insertion and deletion
- No memory wastage like arrays

Types of Linked Lists

---

1. Singly Linked List

Definition

Each node contains **data** and a **pointer to the next node**.

Structure

```
[Data | Next] → [Data | Next] → NULL
```

SPPU TE BE COMP Click to join telegram

Features

- Traversal in **one direction**
- Simple and memory efficient

Example

```
10 → 20 → 30 → NULL
```

---

2. Doubly Linked List

Definition

Each node contains **data**, a **pointer to the previous node**, and a **pointer to the next node**.

Structure

```
NULL ← [Prev | Data | Next] ↔ [Prev | Data | Next] → NULL
```

Features

- Traversal in **both directions**
- Requires extra memory

Example

```
10 ⇄ 20 ⇄ 30
```

---

3. Circular Singly Linked List

Definition

The last node points back to the **first node**, forming a **circle**.

Structure

```
[Data | Next] → [Data | Next]
       ↑_____________________↓
```

Features

- No NULL pointer
- Useful in round-robin scheduling

---

4. Circular Doubly Linked List

Definition

SPPU TE BE COMP Click to join telegram

A doubly linked list where:

- Last node's **next** points to first node
- First node's **previous** points to last node

Structure

```
↔ [Prev | Data | Next] ↔ [Prev | Data | Next] ↔
```

---

2. Operations on singly linked list

Operations on Singly Linked List

A **singly linked list** consists of nodes where each node contains:

- **Data**
- **Link (pointer) to the next node**

```
[Data | Next] → [Data | Next] → NULL
```

---

1. Traversal (Display)

Operation

Visit and display each node from **first (head)** to **last (NULL)**.

Algorithm

1. Set `temp = head`
2. While `temp != NULL` ○ Print `temp->data` ○ `temp = temp->next`

SPPU TE BE COMP Click to join telegram

2. Insertion Operations

a) Insertion at Beginning

Steps

1. Create a new node
2. Set `new->next = head`
3. Update `head = new`

```
Before: 10 → 20 → 30
After : 5 → 10 → 20 → 30
```

---

b) Insertion at End

Steps

1. Create a new node
2. Traverse to the last node
3. Set `last->next = new`
4. Set `new->next = NULL`

```
Before: 10 → 20 → 30
After : 10 → 20 → 30 → 40
```

---

c) Insertion at Specific Position

Steps

1. Traverse to `(position - 1)` node
2. Set `new->next = temp->next`
3. Set `temp->next = new`

---

3. Deletion Operations

a) Deletion at Beginning

Steps

1. Store `head` in `temp`
2. Update `head = head->next`
3. Free `temp`

```
Before: 10 → 20 → 30
After : 20 → 30
```

---

b) Deletion at End

Steps

1. Traverse to second last node
2. Set `secondLast->next = NULL`
3. Free last node

---

c) Deletion at Specific Position

Steps

1. Traverse to `(position - 1)` node
2. Store target node in `temp`
3. Update `prev->next = temp->next`
4. Free `temp`

---

4. Searching

Operation

Find whether a given key exists in the list.

Steps

1. Start from `head`
2. Compare each node's data with key
3. Stop if found or reach NULL

---

5. Counting Nodes

Operation

Count number of nodes in the list.

SPPU TE BE COMP Click to join telegram

Steps

1. Initialize `count = 0`
2. Traverse list and increment count

---

6. Reversing the Linked List

Operation

Reverse the direction of links.

Steps

1. Use three pointers: `prev`, `current`, `next`
2. Reverse links one by one
3. Update `head`

---

3. Doubly linked list operations.

Doubly Linked List Operations

A **doubly linked list (DLL)** is a dynamic data structure in which each node contains:

- **Data**
- **Pointer to the previous node (`prev`)**
- **Pointer to the next node (`next`)**

```
NULL ← [Prev | Data | Next] ↔ [Prev | Data | Next] ↔ NULL
```

---

1. Traversal

a) Forward Traversal

- Move from **head to last node**

Steps

SPPU TE BE COMP Click to join telegram

1. Set `temp = head`
2. While `temp != NULL` o Print `temp->data` o `temp = temp->next`

---

b) Backward Traversal

- Move from **last node to head**

Steps

1. Move `temp` to last node 2.

While `temp != NULL` o Print `temp->data` o `temp = temp->prev`

---

2. Insertion Operations

a) Insertion at Beginning

Steps

1. Create new node `new`
2. `new->prev = NULL`
3. `new->next = head`
4. If `head != NULL`, set `head->prev = new`
5. Update `head = new`

---

b) Insertion at End

Steps

1. Create new node `new`
2. Traverse to last node
3. `last->next = new`
4. `new->prev = last`
5. `new->next = NULL`

SPPU TE BE COMP Click to join telegram

c) Insertion at Specific Position

Steps

1. Traverse to `(position - 1)` node
2. `new->next = temp->next` 3. `new->prev = temp`
4. Update surrounding links:
   - `temp->next->prev = new`
   - `temp->next = new`

---

3. Deletion Operations

a) Deletion at Beginning

Steps

1. Store `head` in `temp`
2. Update `head = head->next`
3. Set `head->prev = NULL`
4. Free `temp`

---

b) Deletion at End

Steps

1. Traverse to last node
2. Update `last->prev->next = NULL`
3. Free `last`

---

c) Deletion at Specific Position

Steps

1. Traverse to target node `temp` 2.

Update links:
   - `temp->prev->next = temp->next`
   - `temp->next->prev = temp->prev`

3. Free `temp`

SPPU TE BE COMP Click to join telegram

4. Searching

Steps

1. Start from `head`
2. Compare each node's data with key
3. Stop if found or reach NULL

---

5. Counting Nodes

Steps

1. Initialize `count = 0`
2. Traverse list and increment count

---

6. Reversing a Doubly Linked List

Steps

1. Swap `prev` and `next` pointers for each node
2. Update `head` to last node

---

4. Circular linked list

Circular Linked List

Definition

A **circular linked list** is a linked list in which the **last node points back to the first node** instead of pointing to `NULL`, forming a **circular structure**.

SPPU TE BE COMP Click to join telegram

Types of Circular Linked List

1. **Circular Singly Linked List**
2. **Circular Doubly Linked List**

---

1. Circular Singly Linked List

Structure

```
[Data | Next] → [Data | Next] → [Data | Next]
       ↑_____________________________________↓
```

Characteristics

- Last node's `next` points to the **head**
- No `NULL` pointer
- Traversal stops when we reach the **head again**

Operations on Circular Singly Linked List

a) Traversal

Steps

1. Set `temp = head`
2. Print `temp->data`
3. Move `temp = temp->next`
4. Repeat until `temp == head`

---

b) Insertion at Beginning

Steps

1. Create new node 2.
If list is empty:
    - `new->next = new` o `head = new`
3. Else:
    - Traverse to last node
    - `new->next = head`
    - o `last->next = new`
    - o Update `head = new`

---

c) Insertion at End

Steps

1. Create new node 2.
   If list is empty:
   - `new->next = new`
   - `head = new`
3. Else:
   - Traverse to last node
   - `last->next = new`
   - `new->next = head`

---

d) Deletion at Beginning

Steps

1. If only one node:
   - Free node and set `head = NULL`
2. Else:
   - Traverse to last node
   - `last->next = head->next`
   - Free old head
   - Update `head`

---

e) Deletion at End

Steps

1. Traverse to second-last node
2. Set `secondLast->next = head`
3. Free last node

---

2. Circular Doubly Linked List

Structure

```
↔ [Prev | Data | Next] ↔ [Prev | Data | Next] ↔
 ↑_____________________________________________↓
```

Characteristics

- Last node's `next` → head
- Head's `prev` → last node

SPPU TE BE COMP Click to join telegram

- Traversal possible in both directions

Operations

- Forward traversal
- Backward traversal
- Insertion at beginning/end
- Deletion at beginning/end

(Operations are similar to doubly linked list with circular links.)

Advantages

- No `NULL` pointers
- Efficient traversal from any node
- Useful for repeating tasks

Applications

- **Round-robin scheduling**
- Multiplayer games
- Music playlists
- Circular queues

---

5. Linked list as ADT.

Linked List as an Abstract Data Type (ADT)

Definition

A **Linked List as an ADT** is a **logical (abstract) representation** of a list in which elements are stored in a **sequence**, and operations are defined **independently of how the list is implemented** (array, singly linked list, doubly linked list, etc.).

👉 ADT focuses on **what operations are performed**, not **how they are implemented**.

Linked List ADT – Concept

In a **Linked List ADT**:

- Data is stored as an **ordered collection of elements**
- Each element has a **position**

- The internal structure (nodes, pointers) is **hidden**

Components of Linked List ADT

1. Data Objects

- A finite ordered set of elements Example:
- `L = {a1, a2, a3, …, an}`

---

2. Operations on Linked List ADT

The following operations are defined logically:

Operation	Description
`create()`	Create an empty list
`isEmpty()`	Check whether list is empty
`insert(pos, x)`	Insert element `x` at position `pos`
`delete(pos)`	Delete element at position `pos`
`search(x)`	Search for element `x`
`traverse()`	Display all elements
`length()`	Return number of elements
`get(pos)`	Get element at position `pos`

ADT Specification of Linked List

Input

- Elements to be stored
- Position (if required)

Output

- Updated list
- Requested element / status

Precondition

- List is initialized

Postcondition

- List reflects insertion or deletion correctly

Example (Logical View)

Consider a list ADT:

`L = {10, 20, 30}` Operations:

- `insert(2, 15)` → `{10, 15, 20, 30}`
- `delete(3)` → `{10, 15, 30}`
- `search(15)` → Found

---

6. Stack ADT definition.

Stack ADT

Definition

A **Stack Abstract Data Type (ADT)** is a **linear data structure** in which elements are inserted and removed **only from one end**, called the **top**. It follows the principle **LIFO (Last In, First Out)**.

Logical View of Stack

```
Top → [ 30 ]
      [ 20 ]
                [ 10 ]
```

Data Objects

- A finite ordered collection of elements
- `S = {a1, a2, a3, …, an}`

Basic Operations of Stack ADT

Operation	Description
`create()`	Create an empty stack
`push(x)`	Insert element `x` onto the top
`pop()`	Remove the top element
`peek()` / `top()`	Return the top element
`isEmpty()`	Check if stack is empty
`isFull()`	Check if stack is full (static stack)
`size()`	Return number of elements

Stack ADT Specification

Precondition

- Stack is initialized

Postcondition

- After `push(x)`: element `x` is at top
- After `pop()`: top element is removed

Example

Initial stack:

`S = {10, 20}` Operations:

- `push(30)` → `{10, 20, 30}`
- `pop()` → `{10, 20}`
- `peek()` → `20`

Applications of Stack ADT

- Function calls (call stack)
- Expression evaluation
- Parenthesis checking
- Undo/Redo operations

7. Implicit vs explicit stack

Implicit Stack	Explicit Stack
Managed by system	Managed by programmer
Used in recursion	Used in iterative solutions
Automatically created	Manually created
No direct access	Direct access
Limited size	Size can be controlled
May cause stack overflow	Overflow can be checked

8. Stack using arrays

Stack Using Arrays

Definition

A **stack using arrays** is an implementation of the **Stack ADT** where stack elements are stored in a **linear array**, and insertion and deletion are performed **only at one end**, called the **top**.
It follows the **LIFO (Last In, First Out)** principle.

Array Representation of Stack

- An array `stack[MAX]` is used
- An integer variable `top` is used to track the top element `o top = -1` → stack is empty `o top = MAX - 1` → stack is full

```
Index:  0   1   2   3
              Stack: [10][20][30][  ]                ↑                top
```

Basic Stack Operations (Array Implementation)

1. Push (Insertion)

Purpose

Insert an element at the top of the stack.

Algorithm

1. If `top == MAX - 1`
   → Stack Overflow
2. Else
   `o top = top + 1 o`
   `stack[top] = element`

C-like Code

```
void push(int x) {     if(top ==
MAX - 1)         printf("Stack
Overflow");     else {
top++;          stack[top] = x;
    }
}
```

2. Pop (Deletion)

Purpose

Remove the top element from the stack.

Algorithm

1. If `top == -1`
   → Stack Underflow

2. Else ○ Remove `stack[top]` ○
   `top = top - 1`

C-like Code

```
int pop() {
if(top == -1) {
        printf("Stack Underflow");
return -1;
    }
    return stack[top--];
}
```

---

3. Peek / Top

Purpose

View the top element without removing it.

```
int peek() {
if(top == -1)
return -1;     return
stack[top];
}
```

---

4. isEmpty

```
int isEmpty() {
return (top == -1);
}
```

---

5. isFull

```
int isFull() {
    return (top == MAX - 1);
                                                                    }
```

Time Complexity

Operation	Time Complexity
Push	O(1)
Pop	O(1)
Peek	O(1)

Advantages

- Simple implementation

- Fast access (O(1))
- Efficient for fixed-size stacks

Disadvantages

- Fixed size → possible **overflow**
- Memory wastage if stack is underused
- Not flexible compared to linked list stack

Applications

- Expression evaluation
- Function calls
- Undo/Redo operations
-

---

9. Stack using linked list

---

tack Using Linked List

Definition

A **stack using linked list** is an implementation of the **Stack ADT** in which stack elements are stored in **nodes of a linked list**, and all **insertions and deletions are performed at one end**, called the **top**.
It follows the **LIFO (Last In, First Out)** principle.

Representation

- Each node contains:
  - **Data**
  - **Link (pointer to next node)**

- The **top** of the stack is represented by the **head pointer**

```
Top → [30 | •] → [20 | •] → [10 | NULL]
```

Basic Stack Operations (Linked List Implementation)

---

1. Push Operation

Purpose

Insert an element at the top of the stack.

Algorithm

1. Create a new node
2. Store data in new node
3. Set `new->next = top`
4. Update `top = new` **C-like Code**

```
void push(int x) {
    struct node* new = (struct node*)malloc(sizeof(struct node));
new->data = x;     new->next = top;     top = new;
}
```

---

2. Pop Operation

Purpose

Remove the top element from the stack.

Algorithm

1. If `top == NULL`
   → Stack Underflow
2. Else
   - Store `top` in temp
   - Update `top = top->next`
   - Free temp

C-like Code

```
int pop() {     if(top == NULL) {
printf("Stack Underflow");
return -1;
    }
```

```
    int x = top->data;
struct node* temp = top;
top = top->next;
free(temp);      return x;
}
```

---

3. Peek / Top

```
int peek() {
if(top == NULL)
return -1;      return
top->data;
}
```

---

4. isEmpty

```
int isEmpty() {
return (top == NULL);
                                                                    }
```

Time Complexity

Operation	Time Complexity
Push	O(1)
Pop	O(1)
Peek	O(1)

Advantages

- Dynamic size (no overflow until memory is full)
- Efficient push and pop (O(1))
- No memory wastage

Disadvantages

- Extra memory required for pointers
- Slightly slower than array stack due to dynamic memory allocation

---

10. Recursion using stack.

Recursion Using Stack

Concept

Recursion is a programming technique in which a function **calls itself** to solve a problem by breaking it into **smaller subproblems**.
Internally, recursion is implemented using a **stack**, known as the **call stack**.

👉 This stack is an **implicit stack**, managed automatically by the system.

How Stack Is Used in Recursion

Whenever a recursive function is called:

1. A **stack frame (activation record)** is created.
2. The stack frame stores:
    - Function parameters
    - Local variables
    - Return address
3. The frame is **pushed onto the stack**.
4. When the function returns, its frame is **popped from the stack**.

This process continues until the **base condition** is reached.

General Flow of Recursion Using Stack

- **Function call** → Push stack frame
- **Recursive call** → Push another frame
- **Base case reached** → Start returning
- **Return** → Pop stack frames one by one

Example: Factorial Using Recursion

Function

```
int fact(int n) {     if(n == 0)          return 1;     return n * fact(n - 1); } Call fact(3);
```

Stack Trace for fact(3)

Stack (Top → Bottom)

fact(0)
fact(1)
fact(2)
fact(3)

Execution

- `fact(3)` calls `fact(2)` • `fact(2)` calls `fact(1)`
- `fact(1)` calls `fact(0)`
- `fact(0)` returns `1`
- Stack frames are popped one by one **Return Calculation**

```
fact(1) = 1 × 1 = 1 fact(2) = 2 × 1 = 2 fact(3) = 3 × 2 = 6
```

Role of Stack in Recursion

Stack Operation	Purpose
Push	Store function call
Pop	Remove completed call
Top	Current executing function

Advantages

- Makes code **simple and readable** • Naturally fits problems like:
  - Tree traversal
  - Factorial ○ Fibonacci ○ Tower of Hanoi

Disadvantages

- Extra **memory overhead** due to stack
- Risk of **stack overflow** if recursion depth is large
- Usually slower than iteration

---

11. Infix to postfix conversion

---

nfix to Postfix Conversion

Definition

Infix notation places operators **between operands** (e.g., `A + B`).

Postfix notation (Reverse Polish Notation) places operators **after operands** (e.g., `AB+`).

Infix → Postfix conversion uses a **stack** to handle operator precedence and associativity.

Why Convert to Postfix?

- No parentheses required
- Easy and fast evaluation using a stack
- Widely used in compilers and calculators

Operator Precedence & Associativity

Operator	Precedence	Associativity
^	Highest	Right to Left
* /	Medium	Left to Right
Operator	**Precedence**	**Associativity**
+ -	Lowest	Left to Right

Algorithm (Using Stack)

Algorithm: InfixToPostfix(expr)

1. Initialize an empty **stack** and empty **postfix string**
2. Scan the infix expression **left to right**
3. If the symbol is an **operand** → append to postfix
4. If the symbol is **(** → push to stack
5. If the symbol is **)** → pop from stack to postfix until `(` is popped 6. If the symbol is an **operator** `op1`:
   - While stack is not empty and `precedence(top) ≥ precedence(op1)`
     → pop from stack to postfix
   - Push `op1` onto stack
7. After scanning, pop all remaining operators from stack to postfix

---

Example

Infix Expression

```
(A + B) * C
```

Step-by-Step Conversion

Symbol	Stack	Postfix
`(`	(	
`A`	(	A
`+`	(+	A
`B`	(+	AB
`)`	—	AB+
`*`	*	AB+
`C`	*	AB+C
End	—	AB+C*

Postfix Expression

`AB+C*`

Another Example

Infix:

```
A + B * C
```

Conversion:

- `B * C` has higher precedence
- Result:

`ABC*+`

Time & Space Complexity

- **Time Complexity:** O(n)
- **Space Complexity:** O(n) (stack)

Applications

- Expression evaluation
- Compiler design
- Calculator implementation

---

12. Evaluate postfix

Evaluation of Postfix Expression

Definition

Postfix expression evaluation is the process of **computing the result** of a postfix (Reverse Polish) expression using a **stack**.

In postfix notation, **operators come after operands**, so **no parentheses** are needed.

Rules for Evaluating Postfix Expression

1. Scan the postfix expression **from left to right** 2. If the symbol is an **operand → push** it onto the stack 3. If the symbol is an **operator**:
    - **Pop two operands** from the stack (second popped = operand2, first popped = operand1) ○ Apply the operator:
      `result = operand1 operator operand2`
    - **Push the result** back onto the stack
4. After scanning the entire expression, the **top of the stack** contains the **final result**

Algorithm

Algorithm: EvaluatePostfix(expr)

1. Initialize an empty stack
2. For each symbol in `expr` ○ If operand → push onto stack ○ If operator →
    - Pop `op2`
    - Pop `op1` Compute `op1 operator op2`
    - Push result
3. Return top of stack

Example 1 (Numerical Expression)

Postfix Expression

```
23*54*+9-
```

Step-by-Step Evaluation

Symbol Stack Action

2 2 Push
3 2, 3 Push
* 6 2 × 3
5 6, 5 Push
4 6, 5, 4 Push
* 6, 20 5 × 4
+ 26 6 + 20

Symbol Stack Action

9 26, 9 Push

- 17 26 − 9

Final Result

```
17
```

Example 2 (Simple Expression)

Postfix:

```
84/2+
```

Evaluation:

- `8 4 /` → `2`
- `2 2 +` → `4`

Result

```
4
```

Time & Space Complexity

- **Time Complexity:** O(n)
- **Space Complexity:** O(n)

Applications

- Expression evaluation in compilers
- Stack-based calculators
- Parsing arithmetic expressions

---

13. Queue ADT

Queue ADT

Definition

A **Queue Abstract Data Type (ADT)** is a **linear data structure** in which elements are **inserted at one end (rear)** and **deleted from the other end (front)**. It follows the principle **FIFO (First In, First Out)**.

Logical View of Queue

```
Front → [10] [20] [30] ← Rear
```

Data Objects

A finite ordered collection of elements:

```
Q = {a1, a2, a3, …, an}
```

Basic Operations of Queue ADT

Operation	Description
`create()`	Create an empty queue
`enqueue(x)`	Insert element `x` at rear
`dequeue()`	Remove element from front
`front()` / `peek()`	View front element
`isEmpty()`	Check if queue is empty
`isFull()`	Check if queue is full (static queue)
`size()`	Return number of elements

Queue ADT Specification

Precondition

- Queue is initialized

Postcondition

- After `enqueue(x)` → element added at rear
- After `dequeue()` → front element removed

Example

Initial Queue:

`Q = {10, 20}` Operations:

- `enqueue(30)` → `{10, 20, 30}`
- `dequeue()` → `{20, 30}`
- `front()` → `20`

Applications of Queue ADT

- CPU scheduling
- Printer spooling
- Breadth First Search (BFS)
- Waiting lines (real-life queues)

---

14. Queue using array.

Queue Using Array

Definition

A **queue using array** is an implementation of the **Queue ADT** in which elements are stored in a **linear array**, with insertion at the **rear** and deletion from the **front**, following the **FIFO (First In, First Out)** principle.

Array Representation of Queue

- An array `queue[MAX]` •
  Two indices:

- `front` → points to the first element
- `rear` → points to the last element

Initial Condition

```
front = -1 rear
= -1
Index:  0   1   2   3
Queue: [10][20][30][  ]
         ↑       ↑       front     rear
```

Basic Queue Operations (Array Implementation)

---

1. Enqueue (Insertion)

Purpose

Insert an element at the rear of the queue.

Algorithm

1. If `rear == MAX - 1`
   → Queue Overflow
2. Else
   - If `front == -1`, set `front = 0` `rear = rear + 1` `queue[rear] = element`

C-like Code

```
void enqueue(int x) {     if(rear
== MAX - 1)         printf("Queue
Overflow");     else {
if(front == -1)             front
= 0;         queue[++rear] = x;
    }
}
```

---

2. Dequeue (Deletion)

Purpose

Remove an element from the front of the queue.

Algorithm

1. If `front == -1` or `front > rear`
   → Queue Underflow
2. Else ○ Remove `queue[front]` ○ `front = front + 1`

C-like Code

```
int dequeue() {
    if(front == -1 || front > rear) {
printf("Queue Underflow");          return
-1;
    }
    return queue[front++];
}
```

---

3. Peek / Front

```
int peek() {
if(front == -1)
return -1;      return
queue[front];
}
```

---

4. isEmpty

```
int isEmpty() {
    return (front == -1 || front > rear); }
```

---

5. isFull

```
int isFull() {
    return (rear == MAX - 1);
                                                                    }
```

Time Complexity

Operation Time Complexity
Enqueue O(1)
Operation Time Complexity
Dequeue O(1)
Peek O(1)

Limitations of Array Queue

- **Fixed size**
- **Wastage of space** after deletions
- Overflow may occur even if free space exists

👉 This limitation is solved using a **Circular Queue**.

Applications

- CPU scheduling
- Printer queue
- BFS traversal

---

15. Queue using linked list.

Queue Using Linked List

Definition

A **queue using linked list** is an implementation of the **Queue ADT** where elements are stored in **nodes of a linked list**.
Insertion is done at the **rear**, and deletion is done from the **front**, following the **FIFO (First In, First Out)** principle.

Representation

- Each node contains: ○ **Data**
  - **Pointer to next node**
- Two pointers are maintained:
  - `front` → points to first node ○ `rear` → points to last node

Front → [10 | •] → [20 | •] → [30 | NULL] ← Rear

Basic Queue Operations (Linked List Implementation)

---

1. Enqueue (Insertion)

Purpose

Insert an element at the rear of the queue.

Algorithm

1. Create a new node
2. Store data in new node
3. Set `new->next = NULL` 4. If queue is empty:
   - `front = rear = new`
5. Else:
   - `rear->next = new` o `rear = new`

C-like Code

```
void enqueue(int x) {
    struct node* new = (struct node*)malloc(sizeof(struct node));
new->data = x;     new->next = NULL;
     if(front == NULL)
front = rear = new;
else {
        rear->next = new;
rear = new;
    }
}
```

---

2. Dequeue (Deletion)

Purpose

Remove an element from the front of the queue.

Algorithm

1. If `front == NULL`
   → Queue Underflow
2. Else:

- Store `front` in temp
- `front = front->next`
- Free temp
- If `front == NULL`, set `rear = NULL`

C-like Code

```
int dequeue() {     if(front == NULL) {          printf("Queue Underflow");          return -1;     }     int x = front->data;
struct node* temp = front;
front = front->next;
if(front == NULL)          rear = NULL;     free(temp);
return x;
}
```

---

3. Peek / Front

```
int peek() {
if(front == NULL)
return -1;     return front->data;
}
```

---

4. isEmpty

```
int isEmpty() {
    return (front == NULL);
}
```

Time Complexity

Operation	Time Complexity
Enqueue	O(1)
Dequeue	O(1)
Peek	O(1)

Advantages

- Dynamic size (no overflow until memory is full)
- No memory wastage
- Efficient insertion and deletion

Disadvantages

- Extra memory required for pointer
- Slightly slower due to dynamic memory allocation

Applications

- CPU scheduling
- BFS traversal
- Printer queue

---

16. Circular queue

Circular Queue

Definition

A **circular queue** is a linear data structure in which the **last position is connected back to the first position**, forming a **circle**.
It follows the **FIFO (First In, First Out)** principle and efficiently utilizes memory by reusing freed spaces.

Why Circular Queue?

In a normal array queue, space gets wasted after deletions.
A circular queue **overcomes this limitation** by allowing the **rear and front to wrap around**.

Representation

- Uses an array `queue[MAX]` • Two pointers: `o front o rear`



Conditions Queue

Empty

`front = -1`

`rear = -1`

Queue Full

`(front == (rear + 1) % MAX)`

Operations on Circular Queue

---

1. Enqueue (Insertion)

Algorithm

1. If queue is full
   → Overflow
2. Else if queue is empty
   - `front = rear = 0`
3. Else
   - `rear = (rear + 1) % MAX`
4. Insert element at `queue[rear]`

---

2. Dequeue (Deletion)

Algorithm

1. If queue is empty
   → Underflow
2. Else if `front == rear` o Set
   `front = rear = -1`
3. Else o `front = (front + 1) % MAX`

---

3. Peek / Front

- Return element at `queue[front]`

C-like Code

```
void enqueue(int x) {
if((rear + 1) % MAX == front)
printf("Queue Overflow");
else if(front == -1) {
front = rear = 0;
queue[rear] = x;
    } else {
        rear = (rear + 1) % MAX;
queue[rear] = x;
    } } int dequeue() {
if(front == -1) {
printf("Queue Underflow");
return -1;
    }
    int x = queue[front];
if(front == rear)
front = rear = -1;     else
        front = (front + 1) % MAX;
return x;
                                                                    }
```

Time Complexity

Operation	Time Complexity
Enqueue	O(1)
Dequeue	O(1)
Peek	O(1)

Advantages

- Efficient memory utilization
- No wastage of space
- Suitable for fixed-size queues

Applications

- CPU scheduling
- Buffer management

- Streaming data
- Traffic light systems

---

17. Deque operations

Deque (Double Ended Queue) Operations

Definition

A **Deque (Double Ended Queue)** is a linear data structure in which **insertion and deletion can be performed at both ends**, i.e., **front and rear**.

👉 A deque generalizes both **stack** and **queue** operations.

Types of Deque

1. **Input-Restricted Deque**
   - Insertion allowed at **one end only**
   - Deletion allowed at **both ends**
2. **Output-Restricted Deque**
   - Deletion allowed at **one end only**
   - Insertion allowed at **both ends**

Basic Deque Operations

---

1. Insert at Front

Operation: `insertFront(x)`

Steps

1. Check overflow

2. Decrement `front`
3. Insert element at `front`

---

2. Insert at Rear

Operation: `insertRear(x)`

Steps

1. Check overflow
2. Increment `rear`
3. Insert element at `rear`

---

3. Delete from Front

Operation: `deleteFront()`

Steps

1. Check underflow
2. Remove element at `front`
3. Increment `front`

---

4. Delete from Rear

Operation: `deleteRear()`

Steps

1. Check underflow
2. Remove element at `rear`
3. Decrement `rear`

---

5. Peek Operations

- `getFront()` → View front element
- `getRear()` → View rear element

---

6. isEmpty

- Returns true if deque is empty

---

7. isFull

- Returns true if deque is full

Deque Representation

```
Front → [10][20][30] ← Rear
```

Time Complexity

Operation	Time Complexity
Insert Front	O(1)
Insert Rear	O(1)
Delete Front	O(1)
Delete Rear	O(1)

Applications

- Sliding window problems
- Palindrome checking
- CPU scheduling
- Undo/Redo systems

---

18. Stack vs queue.

Stack	Queue
Follows **LIFO** principle	Follows **FIFO** principle
Insertion and deletion at **same end (top)**	Insertion at **rear**, deletion at **front**
Uses **push** and **pop** operations	Uses **enqueue** and **dequeue** operations
Has only **one open end**	Has **two open ends**
Can be implemented using array or linked list	Can be implemented using array or linked list
Used in **recursion and function calls**	Used in **scheduling and buffering**

---

19. Queue in networking

Queue in Networking

Definition

In **networking**, a **queue** is a data structure used in **routers, switches, and network interfaces** to **temporarily store packets** when they arrive faster than they can be processed or transmitted.

👉 It follows the **FIFO (First In, First Out)** principle in most basic cases.

Why Queues Are Needed in Networking

- Network traffic is **bursty** (uneven arrival)

- Transmission links have **limited bandwidth**
- Multiple packets may arrive **simultaneously**
- Queues prevent **packet loss** during congestion

Working of Queue in Networking

1. Packets arrive at a **network device**
2. If the output link is busy, packets are placed in a **queue**
3. Packets are transmitted **one by one** based on the queuing policy
4. If the queue is full, **packets may be dropped**

Types of Queues in Networking

1. Input Queue

- Packets wait before being **processed or routed**
- Used in routers and switches

---

2. Output Queue

- Packets wait before being **transmitted**
- Common in network interfaces

---

3. Priority Queue

- Packets are assigned **priorities**
- High-priority packets (voice, video) are sent first

Example: VoIP packets before email packets

---

4. Fair Queuing

- Each flow gets a **fair share of bandwidth**

- Prevents one user from dominating the network

---

5. Round Robin Queue

- Packets are served **cyclically** from multiple queues
- Ensures fairness

Queuing Disciplines (Policies)

Policy	Description
FIFO	First packet served first
Priority Queuing	High-priority first
Weighted Fair Queuing	Bandwidth allocated by weight
Random Early Detection (RED)	Drops packets early to avoid congestion

---

Advantages

- Prevents immediate packet loss
- Smoothens traffic flow
- Improves Quality of Service (QoS)

Disadvantages

- Large queues increase **delay (latency)**
- Can cause **jitter** in real-time traffic
- Queue overflow leads to **packet loss**

Real-Life Example

- Waiting line at a toll booth
- Printer queue
- Data packets waiting in a router

•

Applications

Internet routers
- Switches
- Network congestion control
- Streaming and VoIP services

---

20. Real-time use cases

Real-Time Use Cases (of Stack, Queue, Deque & Related DS)

In **real-time systems**, data structures are used to handle **time-critical tasks**, **data flow**, and **resource management** efficiently. Below are **important real-world use cases**, especially relevant to **Stack and Queue concepts**.

---

1. Stack – Real-Time Use Cases

1. Function Calls (Call Stack)

- Used by the operating system to manage **function execution**
- Stores:
    - Return address ○
      Local variables
- Example: Recursive function execution

2. Undo / Redo Operations

- Text editors (MS Word, VS Code)
- Undo = pop last action
- Redo = push action again

•

3. Expression Evaluation

- Used in calculators and compilers
- Infix → Postfix → Evaluation using stack

4. Backtracking

Maze solving
- Sudoku solver
- Path finding in AI

5. Browser History

- Back button works using stack (LIFO)

---

2. Queue – Real-Time Use Cases

1. CPU Scheduling

- Processes are scheduled in FIFO or priority order
- Used in operating systems

2. Printer Spooling

- Print jobs are queued and executed one by one

3. Network Packet Handling

- Routers use queues to store packets
- Prevents packet loss during congestion

4. Breadth First Search (BFS)

- Used in graphs, routing, social networks

5. Customer Service Systems

- Call centers
- Ticket booking systems

•

3. Circular Queue – Real-Time Use Cases

1. Buffer Management

- Keyboard buffer
- Audio/video streaming buffer

2. Round-Robin Scheduling

- Time-sharing operating systems
- Each process gets equal CPU time

3. Traffic Light Control Systems

- Cyclic control of signals

---

4. Deque – Real-Time Use Cases

1. Sliding Window Problems

- Real-time analytics
- Stock price maximum/minimum

2. Palindrome Checking

- Insert and delete from both ends

3. Task Scheduling

- High-priority tasks from front
- Low-priority tasks from rear

---

5. Linked List – Real-Time Use Cases

1. Dynamic Memory Allocation

- Used in OS memory management

2. Music / Video Playlists

- Easy insertion and deletion

3. Image Viewers

- Next / previous image navigation

---

6. Real-Time Systems Examples (Summary)

Data Structure	Real-Time Use
Stack	Function calls, undo/redo
Queue	CPU scheduling, networking
Circular Queue	Streaming, round-robin

Data Structure	Real-Time Use
Deque	Sliding window, scheduling
Linked List	Dynamic data handling

NUMERICALS

Insert 12 after 10 in linked list {5,10,15,20}.

Given Linked List

```
5 → 10 → 15 → 20 → NULL
```

Operation

Insert **12 after node with value 10**.

Steps

1. Traverse the list to find the node containing **10**
2. Create a new node with data **12**
3. Set `new->next = temp->next` (i.e., point to 15)
4. Set `temp->next = new` (link 10 to 12)

Pointer Adjustment

```
Before:
10 → 15
```

```
After:
                                                    10 → 12 → 15
```

Updated Linked List

```
5 → 10 → 12 → 15 → 20 → NULL
```

Delete last node from doubly linked list 10 25 30 45 60.

Delete the last node from the doubly linked list Given

list:

```
10 ⇄ 25 ⇄ 30 ⇄ 45 ⇄ 60
```

Steps to Delete the Last Node (60)

1. Traverse the list to reach the **last node (60)**.
2. Let `last` point to node **60**.
3. Let `prev` point to node **45** (previous of 60).
4. Set `prev->next = NULL`.
5. Free/delete the node **60**.

Pointer Adjustment

```
Before: 45 ⇄
60 → NULL
After:
45 → NULL
```

Updated Doubly Linked List

10 ⇄ 25 ⇄ 30 ⇄ 45

Convert infix (A+B)(C-D^E) to postfix.*

Convert infix expression to postfix

Given infix expression:

(A+B)∗(C−DE)(A + B) * (C - D^E)(A+B)∗(C−DE)

Operator Precedence

- ^ (highest, right associative)
- * / • + -

Step-by-Step Conversion (Using Stack)

Symbol	Stack	Postfix
(	(	
A	(	A
+	(+	A
B	(+	AB
)	—	AB+
*	*	AB+
(	* (	AB+
C	* (	AB+C
-	* (-	AB+C
D	* (-	AB+CD
^	* (- ^	AB+CD

Symbol	Stack	Postfix
E	* (- ^	AB+CDE
)	*	AB+CDE^-
End	—	AB+CDE^-*

☑ Postfix Expression

```
AB+CDE^-*
```

---

*Evaluate postfix: 6 3 / 4 2 * + 5 -.*

Evaluate Postfix Expression

Given postfix:

```
6 3 / 4 2 * + 5 -
```

Step-by-Step Evaluation (Using Stack)

Symbol	Stack (bottom → top)	Operation
6	6	Push
3	6, 3	Push
/	2	6 ÷ 3 = 2
4	2, 4	Push
2	2, 4, 2	Push
*	2, 8	4 × 2 = 8
+	10	2 + 8 = 10
5	10, 5	Push

Symbol	Stack (bottom → top)	Operation

- 5 10 – 5 = 5

☑ Final Result

5

. Queue operations on size 5: Insert/Delete sequence.

Queue Operations on Size = 5

Initial State

```
Queue size = 5 front = -1, rear = -1 Queue = [ _ , _ , _ , _ , _ ]
```

Operation Sequence

1. Insert (Enqueue) 10

- Queue is empty → set `front = rear = 0`

```
Queue = [10, _ , _ , _ , _ ] front
= 0, rear = 0
```

---

2. Insert (Enqueue) 20

```
Queue = [10, 20, _ , _ , _ ] front
= 0, rear = 1
```

---

3. Insert (Enqueue) 30

```
Queue = [10, 20, 30, _ , _ ] front
= 0, rear = 2
```

---

4. Delete (Dequeue)

- Remove element at `front` → **10**

```
Queue = [ _ , 20, 30, _ , _ ]
front = 1, rear = 2
```

---

5. Insert (Enqueue) 40

```
Queue = [ _ , 20, 30, 40, _ ]
front = 1, rear = 3
```

---

6. Insert (Enqueue) 50

```
Queue = [ _ , 20, 30, 40, 50 ]
front = 1, rear = 4
```

---

7. Delete (Dequeue)

- Remove element at `front` → **20**

```
Queue = [ _ , _ , 30, 40, 50 ] front = 2, rear = 4
```

Final Queue Status

- Elements present:

```
30 → 40 → 50
```

- `front = 2`
- `rear = 4`

Important Exam Note

⚠ Even though the queue has empty spaces at the beginning, **in a normal array queue** no more insertions are possible when `rear = size - 1`. This problem is solved using a **Circular Queue**.

Circular queue operations: Insert/Delete sequence.

Circular Queue Operations (Size = 5)

Initial State

```
MAX = 5
                              front = -1, rear = -1 Queue = [ _ , _ , _ , _ , _ ]
```

Operation Sequence

1. Insert (Enqueue) 10

- Queue empty → set `front = rear = 0`

```
[10, _ , _ , _ , _ ] front
= 0, rear = 0
```

---

2. Insert (Enqueue) 20

```
rear = (0 + 1) % 5 = 1
[10, 20, _ , _ , _ ] front
= 0, rear = 1
```

---

3. Insert (Enqueue) 30

```
rear = (1 + 1) % 5 = 2
[10, 20, 30, _ , _ ] front
= 0, rear = 2
```

---

4. Delete (Dequeue)

- Remove element at `front` → **10**

```
front = (0 + 1) % 5 = 1
[_ , 20, 30, _ , _ ] front
= 1, rear = 2
```

---

5. Insert (Enqueue) 40

```
rear = (2 + 1) % 5 = 3 [_
, 20, 30, 40, _ ] front =
1, rear = 3
```

---

6. Insert (Enqueue) 50

```
rear = (3 + 1) % 5 = 4 [_
, 20, 30, 40, 50 ] front
= 1, rear = 4
```

---

7. Delete (Dequeue)

- Remove element at `front` → **20**

```
front = (1 + 1) % 5 = 2 [_
, _ , 30, 40, 50 ] front =
2, rear = 4
```

---

8. Insert (Enqueue) 60 ☑ (Wrap-Around)

```
rear = (4 + 1) % 5 = 0 [60, _ , 30, 40, 50 ] front = 2, rear = 0
```

Final Queue (Logical Order)

```
30 → 40 → 50 → 60
```

Important Conditions (Exam Point)

- **Queue Full:**

(front==(rear+1)%MAX)(front == (rear + 1) \% MAX)(front==(rear+1)%MAX)

- **Queue Empty:** `front = rear = -1`